The next step is to adjust the `createSpace` method to use the `CapabilityController` to create a capability URI to return, as shown in listing 9.3. The code changes are very minimal; simply call the `createUri` method to create the capability URI. As the user that creates a space is given full permissions over it, you can pass in all permissions when creating the URI. Then use the `uri.toASCIIString()` method to convert the URI into a properly encoded string. Because you're going to use capabilities for access you can remove the lines that insert into the `user_roles` table; these are no longer needed. Open SpaceController.java in your editor and adjust the implementation of the `createSpace` method to match listing 9.3. New code is highlighted in bold.

Listing 9.3 Returning a capability URI

```
public JSONObject createSpace(Request request, Response
response) {
var json = new JSONObject(request.body());
var spaceName = json.getString("name");
if (spaceName.length() > 255) {
throw new IllegalArgumentException("space name too long");
}
var owner = json.getString("owner");
if (!owner.matches("[a-zA-Z][a-zA-Z0-9]{1,29}")) {
throw new IllegalArgumentException("invalid username");
}
var subject = request.attribute("subject");
if (!owner.equals(subject)) {
throw new IllegalArgumentException(
"owner must match authenticated user");
}
return database.withTransaction(tx -> {
var spaceId = database.findUniqueLong(
"SELECT NEXT VALUE FOR space_id_seq;");
```